

Vision-Language Model Implementation and Evaluation Report

1 Problem

The objective of this project was to build a simple Vision-Language Model (VLM) from scratch capable of interpreting images containing basic geometric shapes and generating accurate text descriptions of their sizes, colors, shapes, and spatial relations. The model must process an input image and output a tokenized sequence describing it. My primary goal for this project was to learn the inner workings of multimodal architectures, specifically how a convolutional encoder can be combined with a transformer decoder using handwritten multi-head attention mechanisms. By building this without relying on pre-packaged large language model libraries, I wanted to understand the practical challenges of tensor shape alignment, causal masking, and preventing data leakage during sequence generation.

2 Architecture

The architecture follows a standard encoder-decoder multimodal pipeline designed for simplicity and educational transparency. The key design choice was to keep the components modular so they could be verified individually before moving up the pipeline.

- **Tokenizer (tokenizer.py):** Converts raw text into a sequence of integer tokens. It uses a small, fixed vocabulary containing specific sizes (small, large), colors (red, green, blue, yellow), shapes (circle, square, triangle, cross), and relations (leftof, rightof, above, below).
- **Vision Encoder (encoder.py):** Acts as the feature extractor. It takes an input image tensor of shape [Batch, Channels, Height, Width] and processes it through convolutional layers to produce a flattened spatial feature map of shape [Batch, 64, 64] (where 64×64 represents the spatial grid flattened into a sequence).
- **Adapter:** A linear projection layer that maps the vision encoder's feature dimension to the transformer's hidden dimension size, resulting in a shape of [Batch, Num_Patches, Hidden_Dim].
- **Transformer Decoder (decoder.py & attention.py):** A custom-written decoder utilizing Multi-Head Attention (MHA), a Multi-Layer Perceptron (MLP), and Layer Normalization (LN). It processes the token embeddings (shape [Batch, Sequence_Length, Hidden_Dim]) and cross-attends to the visual features.
- **VLM Wrapper (vlm.py):** Combines the CNN, adapter, and decoder into a single trainable model.

I chose this specific setup because it mirrors the core concepts of models like LLaVA but at a vastly reduced scale, allowing it to be trained and debugged on standard consumer hardware.

3 Implementation

Several non-obvious implementation details were required to ensure the model trained correctly, particularly regarding attention masking and debugging.

- **Attention and Masking:** In the attention module, I implemented a causal look-ahead mask for the self-attention mechanism in the decoder. This ensures that when predicting the token at position t , the model can only attend to tokens at positions 0 to $t - 1$. Without this boolean mask (where future positions are set to negative infinity before the softmax layer), the model would simply copy the target sequence during training and completely fail during generation.
- **Loss Masking:** During the loss calculation in the training loop, I applied masking to the padding tokens. Because sequences vary in length, shorter sequences are padded. The Cross-Entropy loss is configured to ignore the padding token index so that the model is not penalized for or incentivized to predict arbitrary padding.
- **Correctness Checking:** Before running the full training loop, I implemented a sanity check to verify the pipeline. The goal was to overfit a single batch of data. This is a standard bug check in deep learning to ensure gradients flow correctly and the loss can decrease. Over 99 epochs on a single batch, the loss successfully dropped from 3.437907 to 0.007649, confirming that the model architecture and optimization steps were functioning properly.

4 Training Setup

Parameter	Value
Optimizer	AdamW
Learning Rate	3e-4
Batch Size	32
Epochs	14 (Saved best checkpoint based on validation loss)
Hardware	AMD Ryzen 5 7520U CPU with Radeon Graphics (No CUDA available)
Environment	Windows 11, Python 3.12.0, PyTorch 2.4.1+cpu
Seed	42

Table 1: Hyperparameters and environment setup.

5 Experiments

The experiments were designed to test the model's ability to not only memorize the training data but to generalize to new, unseen configurations of shapes, colors, and sizes.

- **Question:** Can a simple, scratch-built VLM learn to accurately parse multi-object images and understand spatial relations using a small dataset and CPU training?

- **Hypothesis:** The model will learn to identify individual attributes (like color and size) relatively quickly but will struggle with relational logic (e.g., "left of", "above") and will face generalization issues on completely unseen combinations.
- **What stayed fixed:** The model architecture, the tokenizer vocabulary, the learning rate, and the hardware environment (AMD Ryzen CPU) remained constant across all evaluations.
- **What changed (The Data Splits):** I evaluated the best saved checkpoint (checkpoints/best.pt) across three different data distributions:
 1. **Standard Test Split:** 2000 samples drawn from the same distribution as the training set.
 2. **Held-out Test Split:** 2000 samples featuring novel combinations of shapes and colors that were explicitly withheld during training.
 3. **Blind Test Split:** A completely distinct set of 2000 samples to verify true out-of-distribution performance.

Additionally, I ran a throughput benchmark to measure the processing speed of my hardware setup during training.



Figure 1: S1 Training Throughput on AMD Ryzen 5 CPU. Notice the processing speed is limited due to relying solely on CPU computation, resulting in longer training times per epoch.

6 Results

The model was evaluated using a custom evaluation script. All metrics are presented as percentage accuracy (%).

Metric	Test Split	Test Held-out Split	Blind Test Split
Exact Match	70.35%	0.10%	1.65%
Size Accuracy	84.14%	83.76%	33.28%
Color Accuracy	74.40%	76.44%	20.10%
Shape Accuracy	74.24%	26.14%	20.63%
Relation Accuracy	43.43%	47.37%	0.00%
Unparsed Predictions	1 / 2000	18 / 2000	0 / 2000

Table 2: Evaluation Metrics across different dataset splits (evaluated on AMD Ryzen CPU). Notice the severe drop in "Exact Match" and "Shape Accuracy" when moving from the standard test set to the held-out test set, indicating severe overfitting to training combinations.

7 Analysis

The evidence strongly supports the hypothesis that the model easily memorizes basic attributes but fails to generalize. On the standard test split, the model achieved a solid 70.35% Exact Match, meaning it perfectly described the entire image in 7 out of 10 cases. Its ability to recognize Size (84.14%) and Color (74.40%) was robust.

However, the evidence highlights a massive generalization gap. When tested on the `test_heldout` split, the Exact Match plummeted to 0.10%. Interestingly, while Size and Color accuracy remained high (83.76% and 76.44%), Shape accuracy crashed from 74.24% to 26.14%. This suggests the model did not learn the true visual features of the shapes; instead, it memorized specific color-shape combinations from the training set. When presented with a novel combination (e.g., a color it had never seen applied to a triangle), the visual encoder failed to recognize the shape entirely.

Furthermore, Relation Accuracy was poor across the board, peaking at only 47.37%. This indicates that the 64×64 feature map from the CNN, combined with the handwritten multi-head attention, struggles to model the spatial geometry required to understand "left of" or "above". The Blind Test confirmed these weaknesses, yielding only a 1.65% Exact Match and a 0.00% Relation Accuracy.

8 Limitations

The most significant limitation of this project was the hardware constraints. Training was conducted entirely on an AMD Ryzen 5 CPU because CUDA was not available. This severely restricted the batch size and the number of epochs I could realistically run, preventing me from testing larger hidden dimensions or deeper transformer layers. I also did not test any data augmentation techniques, which likely contributed to the extreme overfitting observed in the held-out data. I also did not implement dropout in the transformer blocks. The low relation accuracy could be wrong due to how I generate the spatial coordinate embeddings in the CNN; perhaps they are not distinct enough for the attention mechanism to utilize. If I were to do this next, I would implement dropout, add random color jitter and cropping to the dataset to force the model to learn shapes independently of colors, and run the training on a cloud GPU for at least 50 epochs to allow the spatial relations to converge properly.

9 References

- Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems. (Used for implementing the handwritten Multi-Head Attention equations).
- PyTorch Documentation (2024). *torch.nn modules*. (Used for implementing Layer-Norm, CrossEntropyLoss, and tensor permutations).